

v1.8 — Search, Notifications, Calendars, Reporting & Collaboration Specification

Absolutely. Building on v1.7, the next specification should define the **cross-case information and collaboration layer**: finding information quickly, notifying users about work that needs attention, integrating calendars, producing operational/legal-management reports, and enabling controlled collaboration.

ILCMS — Technical Specification v1.8

Search, Notifications, Calendars, Reporting & Collaboration Specification

Project: Icelandic Legal Case Management System (ILCMS)

Specification Version: 1.8

Status: Technical Design Baseline

Depends on: v1.1–v1.7

Primary Areas: Search, Notifications, Calendars, Reporting, Collaboration

1. Purpose

This specification defines the architecture and implementation requirements for the ILCMS information-discovery, notification, calendar, reporting, and collaboration capabilities.

The components defined here provide the mechanisms by which users can:

- find cases, documents, parties, proceedings and other information;
- receive timely notifications about work and events;
- manage hearings, meetings and deadlines through calendars;
- synchronize selected events with external calendar systems;
- produce operational and management reports;
- collaborate securely within cases and teams;
- monitor workloads and outstanding work;
- subscribe to relevant events without receiving unauthorized information.

The design must integrate with:

- **v1.5 — Authentication, Authorization & Security**
- **v1.6 — Document Management, Electronic Records & Archiving**
- **v1.7 — Case Management, Workflow, Tasks, Deadlines & Legal Process**

- **v1.4 — REST API & OpenAPI**
- **v1.3 — Database & Data Model**
- **v1.1 — DevOps, CI/CD & Kubernetes**

2. Design Principles

2.1 Authorization is mandatory

Search results, notifications, reports, calendar entries and collaboration features must respect the authorization model defined in v1.5.

A user must never discover information merely because they can search for it.

This applies to:

- case metadata;
- documents;
- parties;
- tasks;
- deadlines;
- communications;
- notes;
- proceedings;
- reports;
- notifications;
- calendar events.

Authorization must be evaluated server-side.

2.2 Search is not an authorization bypass

The search index must not become an alternative security boundary that exposes information hidden by the application.

Every search request must be authorization-aware.

Where practical, authorization constraints should be incorporated into the search query itself rather than retrieving unrestricted results and filtering them afterward.

2.3 Notifications are not authoritative records

Notifications are derived from authoritative database state.

For example:

Deadline



Notification rule



Notification event



Delivery



User notification

The notification is informational.

The underlying deadline remains authoritative.

2.4 External calendars are secondary systems

The ILCMS database remains authoritative for:

- hearings;
- legal deadlines;
- case events;
- internal tasks;
- court events.

External calendars are synchronization targets.

An external calendar failure must not modify or invalidate the authoritative ILCMS record.

2.5 Reports must respect access controls

A report must not provide aggregate information that allows users to infer information about cases they cannot access.

Authorization must therefore apply both to:

1. the underlying records;
2. the generated report.

3. Search Architecture

3.1 Search objectives

The search subsystem shall provide fast full-text and structured search across:

- cases;
- documents;
- document contents;
- parties;
- users;
- organizations;
- proceedings;
- court case numbers;
- tasks;
- deadlines;
- hearings;
- communications;
- notes where permitted;
- case events.

4. Search Types

ILCMS shall support three primary search mechanisms.

4.1 Exact lookup

Used when the user knows a specific identifier.

Examples:

M-2026-00125

E-1234/2026

document UUID

Exact identifiers should be indexed for very fast lookup.

4.2 Structured search

Structured search uses explicit fields.

Examples:

```
Case type = CIVIL
Status = ACTIVE
Responsible lawyer = user123
Opened after = 2026-01-01
Opened before = 2026-09-01
```

4.3 Full-text search

Full-text search should support:

- case titles;
- document metadata;
- OCR text;
- document text;
- correspondence;
- permitted notes;
- party names;
- classification metadata.

Search must support Icelandic text.

5. Icelandic Language Search

The search engine must support Icelandic UTF-8 text.

Search configuration should account for:

- Icelandic characters;
- case-insensitive searching;
- Unicode normalization;
- tokenization;
- stemming/lemmatization where supported;

- common Icelandic inflection;
- names;
- legal terminology.

The implementation must not assume English-only linguistic processing.

Search behavior should be configurable so that an initial implementation can use PostgreSQL full-text search and later migrate to a dedicated search engine if required.

6. Search Technology

The initial implementation should support one of:

Option A — PostgreSQL

Suitable for MVP deployments.

Possible technologies:

- PostgreSQL `tsvector`;
- GIN indexes;
- trigram indexes using `pg_trgm`;
- full-text search configurations.

Option B — Dedicated Search Engine

For larger deployments:

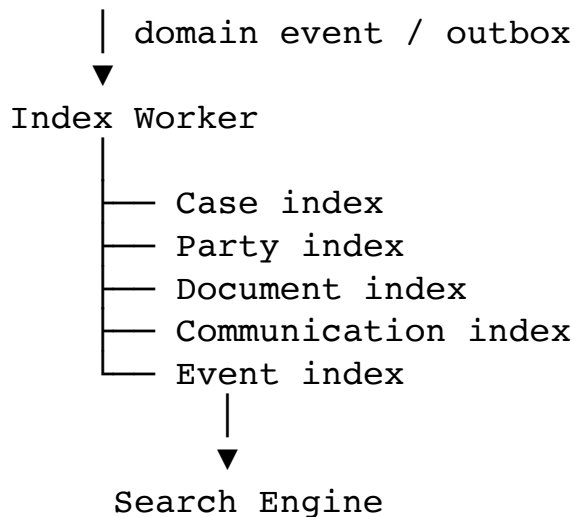
- OpenSearch;
- Elasticsearch-compatible infrastructure;
- another internally hosted search engine.

The application should use a search abstraction so that the underlying implementation can be replaced without changing the public API.

7. Search Index Architecture

A typical architecture is:

PostgreSQL
|



The search index is a derived data store.

PostgreSQL remains authoritative.

8. Search Index Security

Search documents must contain sufficient information to enforce authorization.

Possible indexed authorization attributes include:

organization_id
team_ids
authorized_user_ids
confidentiality_level
case_id

However, highly sensitive access-control information should not be unnecessarily duplicated.

The preferred approach is:

1. identify authorized case IDs;
2. apply authorization filters to the search query;
3. retrieve matching records;
4. perform final object authorization before returning results.

9. Search API

Example:

GET /api/v1/search?q=Einarsson

Structured search:

```
GET /api/v1/search?
    q=contract&
    type=document&
    case_type=CORPORATE&
    status=ACTIVE
```

Response:

```
{
  "items": [
    {
      "type": "case",
      "id": "uuid",
      "case_number": "M-2026-00125",
      "title": "Example matter",
      "highlight": "..."
    }
  ],
  "total": 1,
  "page": 1,
  "page_size": 25
}
```

10. Search Result Types

Each result must identify its resource type.

Example:

```
CASE
DOCUMENT
PARTY
TASK
DEADLINE
HEARING
PROCEEDING
COMMUNICATION
NOTE
```

Results should contain enough context to allow the frontend to navigate directly to the authorized resource.

11. Search Ranking

Search results should support relevance ranking.

Ranking may consider:

- exact case-number match;
- exact party-name match;
- title match;
- metadata match;
- document-content match;
- recency;
- term frequency;
- configured business weighting.

Exact case-number matches should normally rank above ordinary full-text matches.

12. Search Highlighting

For full-text results, the API may return highlighted matching fragments.

Example:

```
{  
  "highlight": "...samningur um <mark>kaupsamning</mark>..."  
}
```

The frontend must safely render search highlights.

Search results must never inject unsanitized HTML.

13. Advanced Search

The UI should provide an advanced-search interface supporting combinations of:

- case number;
- title;
- client;
- opposing party;
- lawyer;
- team;
- classification;
- case type;

- case status;
- confidentiality;
- document type;
- document date;
- case opening date;
- deadline;
- court;
- court case number;
- proceeding;
- tags.

Filters should be composable.

14. Saved Searches

Users may save frequently used searches.

Example:

```
My active civil cases
Cases with deadlines within 14 days
Unassigned cases
Cases awaiting court response
Saved searches should contain:
```

```
id
user_id
organization_id
name
query_definition
created_at
updated_at
```

Search definitions must be versioned if the search syntax changes.

15. Notification Architecture

Notifications provide users with actionable information.

Examples:

- deadline approaching;
- task assigned;
- task overdue;
- document received;
- document processing completed;
- hearing approaching;
- case assigned;
- case status changed;
- approval required;
- conflict review required;
- court event approaching.

16. Notification Channels

ILCMS should support:

In-app notifications

Primary notification mechanism.

Email

Optional and configurable.

Push notifications

Future capability.

External integrations

Possible future channels:

- Microsoft Teams;
- Slack;
- enterprise messaging systems.

External integrations must be optional.

17. Notification Model

Suggested table:

```
notifications
-----
id
organization_id
user_id
type
priority
title
message
case_id
task_id
deadline_id
hearing_id
document_id
event_id
created_at
read_at
delivered_at
delivery_status
```

18. Notification Types

Recommended enumeration:

```
TASK_ASSIGNED
TASK_DUE_SOON
TASK_OVERDUE
```

```
DEADLINE_CREATED
DEADLINE_CHANGED
DEADLINE_DUE_SOON
DEADLINE_OVERDUE
```

```
CASE_ASSIGNED
CASE_STATUS_CHANGED
```

```
DOCUMENT_RECEIVED
DOCUMENT_PROCESSING_COMPLETE
DOCUMENT_PROCESSING_FAILED
```

HEARING_SCHEDULED
HEARING_APPROACHING
HEARING_CHANGED

APPROVAL_REQUIRED
CONFLICT_REVIEW_REQUIRED

COURT_EVENT_CREATED
COURT_EVENT_APPROACHING

LEGAL_HOLD_CREATED
LEGAL_HOLD_CHANGED

19. Notification Preferences

Users must be able to configure notification preferences.

Example:

Notification type

↓

In-app: ON

Email: ON

Push: OFF

Preferences may be configured by:

- notification type;
- channel;
- priority.

Administrators may define organization-wide mandatory notifications.

Users must not be able to disable legally/operationally mandatory system alerts where organizational policy requires them.

20. Notification Deduplication

The notification system must prevent duplicate notifications.

Each generated notification should have an idempotency key.

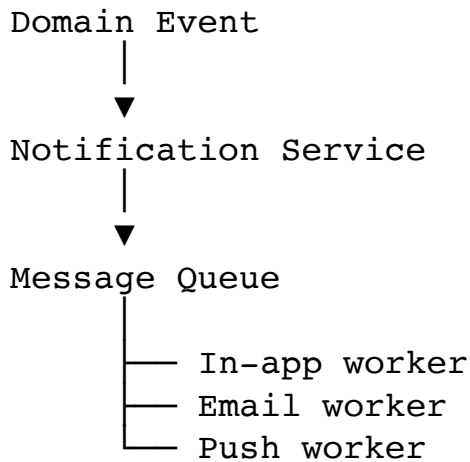
Example:

`deadline_id + notification_type + threshold + user_id`

This prevents scheduler retries from generating duplicate notifications.

21. Notification Delivery

Recommended architecture:



Delivery should support:

- retries;
- exponential backoff;
- dead-letter queues;
- delivery status;
- operational metrics.

22. Notification Read State

Users must be able to:

GET /api/v1/notifications

Mark one as read:

POST /api/v1/notifications/{id}/read

Mark all as read:

POST /api/v1/notifications/read-all

Unread count:

GET /api/v1/notifications/unread-count

23. Notification Security

Notification messages must not disclose information beyond the user's authorization.

Avoid messages such as:

Confidential case M-2026-00125 involving [sensitive party]
when the notification could appear in:

- email previews;
- mobile notifications;
- lock-screen notifications.

External notifications should contain minimal information.

24. Calendar Architecture

The calendar subsystem provides a unified view of:

- hearings;
- trials;
- meetings;
- mediation;
- interviews;
- deadlines;
- tasks;
- court events;
- selected organizational events.

25. Calendar Views

The frontend should support:

- day;
- week;
- month;
- agenda/list;
- case calendar;
- personal calendar;

- team calendar.

26. Calendar Event Categories

Events should distinguish between:

HEARING
TRIAL
CASE_MANAGEMENT
MEDIATION
SETTLEMENT
INTERVIEW
MEETING
VIDEO_CALL
DEADLINE
TASK
COURT_EVENT
OTHER

27. Calendar Event Model

Suggested table:

```
calendar_events
-----
id
organization_id
case_id
event_type
title
description
location
start_at
end_at
timezone
all_day
status
created_by
created_at
updated_at
```

Where the event represents an existing authoritative object, the record should also retain a reference:

```
source_type
```

`source_id`

For example:

`source_type = HEARING`

`source_id = <hearing UUID>`

28. Time Zone Handling

All timestamps must be timezone-aware.

The system should use:

UTC

for canonical storage where appropriate, while retaining the intended local timezone for calendar events.

For Iceland:

Atlantic/Reykjavik

must be supported.

The frontend should render dates and times according to the user's configured timezone unless an event explicitly requires another timezone.

29. Recurring Events

Recurring meetings may be supported for administrative events.

Examples:

Weekly team meeting

Monthly management meeting

Legal deadlines must **not** be represented merely as generic recurring calendar events.

Legal deadline generation must continue to use the deadline engine defined in v1.7.

30. Calendar Conflict Detection

The system should detect obvious scheduling conflicts.

Example:

09:00–10:00 Hearing A

09:30–10:30 Hearing B

The system should warn the responsible user.

Conflict detection is advisory and must not automatically modify court or legal events.

31. External Calendar Integration

The architecture should support adapters for:

- Microsoft 365 / Outlook;
- Google Calendar;
- CalDAV;
- other enterprise calendar systems.

Integration should be implemented behind a common interface.

Example:

```
class CalendarProvider:
    create_event()
    update_event()
    delete_event()
    list_events()
    sync()
```

32. External Calendar Synchronization

Synchronization should support:

ILCMS → External calendar

External calendar → ILCMS

However, not every event should be synchronized bidirectionally.

Recommended initial model:

Legal deadlines:

ILCMS → external calendar

Court hearings:

ILCMS → external calendar

Internal meetings:

configurable bidirectional synchronization

33. External Calendar Privacy

External calendar entries should contain minimal confidential information.

Instead of:

Criminal case involving John Doe — evidence review
prefer:

ILCMS — Hearing

Case: M-2026-00125

or, where appropriate:

ILCMS — Confidential Case Event

The exact policy should be configurable.

34. Calendar Synchronization State

Synchronization should maintain:

calendar_sync

id

user_id

provider

external_calendar_id

external_event_id

internal_event_id

sync_direction

last_synced_at

sync_status

error_message

External IDs must never replace ILCMS primary identifiers.

35. Calendar Failure Handling

If synchronization fails:

1. retain the ILCMS event;
2. record synchronization failure;
3. retry;
4. notify administrators/user if necessary;
5. provide synchronization status.

An external calendar outage must not affect legal deadline calculation.

36. Reporting Architecture

The reporting subsystem provides:

- operational reports;
- workload reports;
- case statistics;
- deadline reports;
- task reports;
- court/proceeding statistics;
- document statistics;
- management dashboards.

37. Reporting Principles

Reports must be:

- authorization-aware;
- reproducible;
- timestamped;
- auditable;
- filterable;
- exportable;
- performant.

Reports must clearly distinguish between:

`Current state`
and:

`Historical/event-based statistics`

38. Core Reports

The MVP should support:

Case report

Cases by:

- status
- type
- classification
- lawyer
- team
- opening date
- closing date

Deadline report

Upcoming deadlines

Overdue deadlines

Deadlines by responsible lawyer

Deadlines by case

Task report

Open tasks

Overdue tasks

Tasks by lawyer

Tasks by team

Completed tasks

Workload report

Open cases per lawyer

Open tasks per lawyer

Upcoming deadlines per lawyer

Upcoming hearings per lawyer

39. Case Aging

Case aging should be calculated from case history.

Possible metrics:

days since opened

days active

days pending

days since last activity

The system should distinguish:

Total case age

from:

Active working time

if the workflow history supports such calculation.

40. Court and Proceeding Reports

Reports should support:

- proceedings by court;
- cases by proceeding type;
- active proceedings;
- completed proceedings;
- appeals;
- upcoming hearings;
- court deadlines;
- decisions/judgments.

41. Report Filters

Reports should support filters such as:

organization

department

team

lawyer

case type

classification

case status

court

date range

priority

confidentiality

Only fields for which the user has authorization should be available.

42. Report Exports

Reports should support:

CSV

XLSX

PDF
depending on report type.

Large exports should be asynchronous.

Example:

POST /api/v1/reports/cases/export
Response:

202 Accepted
with a job ID.

The user can then retrieve the generated file.

43. Report Auditability

Reports involving sensitive legal information should record:

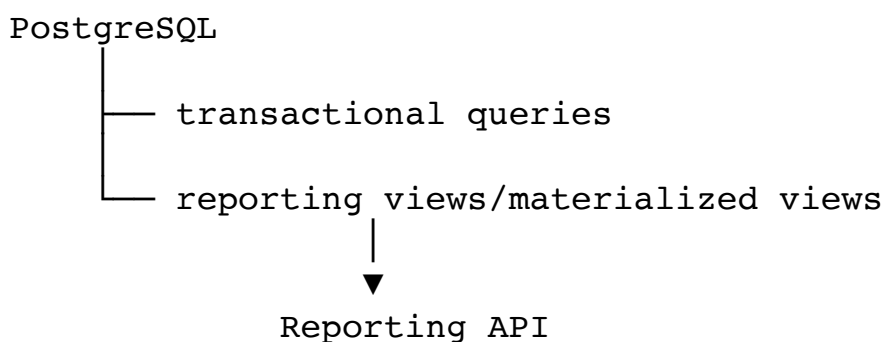
user_id
organization_id
report_type
filters
generated_at
export_format
result_count

Particularly sensitive reports may require additional audit logging.

44. Reporting Performance

Reports must not execute unrestricted expensive queries against production transactional tables.

Possible architecture:



For larger installations, a dedicated reporting database may be introduced.

45. Collaboration Model

Collaboration is primarily case-centric.

Users collaborate through:

- assignments;
- tasks;
- documents;
- comments/notes;
- communications;
- approvals;
- timeline events;
- shared case activity.

The system should avoid creating an unrestricted generic chat environment that bypasses case records management.

46. Case Activity Feed

Each case should provide an activity feed.

Example:

09:32 — Document uploaded

09:45 — Task assigned to Lawyer A

10:10 — Deadline changed

11:30 — Hearing scheduled

14:20 — Lawyer B completed task

Events originate from the authoritative event stream defined in v1.7.

47. Comments and Collaboration Notes

Where appropriate, users may comment on:

- tasks;
- documents;
- workflow steps;

- case activities.

Comments should contain:

id
parent_type
parent_id
author_id
body
created_at
updated_at
deleted_at

Deletion should preferably be logical rather than physical where auditability requires preservation.

48. Comment Visibility

Visibility should support:

CASE_TEAM
ORGANIZATION
PRIVATE

The existing v1.7 note model remains authoritative.

Private comments must never become visible merely because another user gains access to the case.

49. Document Collaboration

Document collaboration must integrate with v1.6.

Users should be able to:

- upload documents;
- view document metadata;
- create versions;
- add comments;
- assign document-related tasks;
- review documents;
- approve documents where configured.

The system must preserve document version history.

50. Document Review Workflow

Optional review workflow:

DRAFT

↓

IN_REVIEW

↓

CHANGES_REQUESTED

↓

APPROVED

↓

FINAL

The workflow must be configurable.

Approval does not automatically constitute a legal signature unless an approved electronic-signature mechanism is used.

51. Presence and Collaboration Indicators

The UI may show:

User currently viewing case

User editing document

Recently active users

These indicators are informational only.

They should not expose private activity outside the user's authorized context.

52. Real-Time Updates

ILCMS may support real-time updates using:

- WebSockets;
- Server-Sent Events;
- polling fallback.

Example:

Lawyer A assigns task

↓

Backend event

↓

Notification/event service

↓

Lawyer B UI updates

Real-time functionality must not be required for correctness.

53. Optimistic Concurrency

Collaboration must use the concurrency controls defined in v1.4.

For mutable resources:

ETag

If-Match

version number

may be used.

Example:

If-Match: "version-17"

If another user changed the object:

409 Conflict

must be returned.

54. Subscriptions

Users may subscribe to selected case events.

Examples:

Notify me when:

- a document is added;
- a deadline changes;
- a court event is created;
- a task is completed.

Subscriptions should be explicit and authorization-aware.

Suggested table:

subscriptions

id

user_id

organization_id

resource_type

resource_id

event_type
created_at
enabled

A subscription must automatically stop generating events if the user loses access to the underlying resource.

55. Collaboration Notifications

Examples:

"You have been assigned task X."

"Document Y is ready for review."

"Deadline Z was changed."

"A new court event was added to case M-2026-00125."

Notifications should link directly to the authorized resource.

56. Dashboard Architecture

The main dashboard should combine information from v1.7 and v1.8.

Recommended widgets:

My Tasks
Upcoming Deadlines
Overdue Tasks
Upcoming Hearings
Recent Cases
Recent Documents
Unread Notifications
Recent Activity
Workload

Users should be able to configure dashboard widgets where practical.

57. Team Dashboard

Team managers may receive:

Open cases
Unassigned cases
Cases by lawyer

Overdue tasks
Upcoming deadlines
Upcoming hearings
Workload distribution
Recently opened cases
Recently closed cases
Access must depend on organizational role.

58. Management Dashboard

Organization administrators may receive aggregated:

Cases by type
Cases by status
Case aging
Workload
Deadline performance
Task completion
Proceedings
Document volumes
System activity
Highly confidential case information should not automatically appear in management dashboards.

59. Metrics

Operational metrics should include:

cases_open
cases_closed
tasks_open
tasks_overdue
deadlines_upcoming
deadlines_overdue
hearings_upcoming
notifications_pending
notifications_failed
documents_pending_processing
search_latency
report_generation_time
calendar_sync_failures
Metrics must not contain unnecessary personal or case-sensitive content.

60. Search Index Consistency

Because search is eventually consistent, the system must define expected behavior.

After a successful mutation:

PostgreSQL = immediately authoritative

Search index = eventually consistent

The UI should not imply that a successful database transaction necessarily means the object is immediately searchable.

61. Reindexing

Administrators must be able to rebuild indexes.

Example:

POST /api/v1/admin/search/reindex

Large reindex operations must run asynchronously.

Requirements:

- progress tracking;
- resumability;
- throttling;
- authorization;
- audit logging;
- no interruption to normal case management.

62. Notification Scheduler

The scheduler defined in v1.7 must execute:

deadline reminder generation

task reminder generation

hearing reminders

subscription events

notification cleanup

failed delivery retries

Scheduler jobs must be idempotent.

63. Calendar Synchronization Scheduler

Calendar synchronization should run independently of deadline processing.

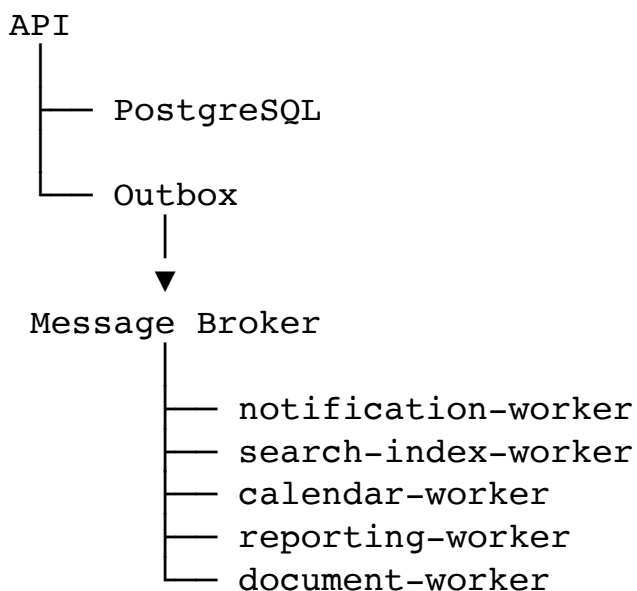
Example:

`calendar-sync-worker`
should handle:

- outbound event synchronization;
- inbound synchronization where enabled;
- token refresh;
- conflict detection;
- retries;
- expired credentials.

64. Background Job Architecture

Recommended:



Workers must be independently scalable.

65. API Endpoints

Search

GET /api/v1/search
GET /api/v1/search/saved
POST /api/v1/search/saved
GET /api/v1/search/saved/{id}
PUT /api/v1/search/saved/{id}
DELETE /api/v1/search/saved/{id}

Notifications

GET /api/v1/notifications
GET /api/v1/notifications/unread-count
POST /api/v1/notifications/{id}/read
POST /api/v1/notifications/read-all
GET /api/v1/notification-preferences
PUT /api/v1/notification-preferences

Calendar

GET /api/v1/calendar/events
POST /api/v1/calendar/events
GET /api/v1/calendar/events/{id}
PUT /api/v1/calendar/events/{id}
DELETE /api/v1/calendar/events/{id}

Calendar integrations

GET /api/v1/calendar/providers
POST /api/v1/calendar/connections
DELETE /api/v1/calendar/connections/{id}
POST /api/v1/calendar/connections/{id}/sync
GET /api/v1/calendar/connections/{id}/status

Reports

GET /api/v1/reports/cases
GET /api/v1/reports/tasks
GET /api/v1/reports/deadlines
GET /api/v1/reports/workload
GET /api/v1/reports/proceedings
POST /api/v1/reports/{report_type}/export
GET /api/v1/reports/jobs/{id}

Collaboration

GET /api/v1/cases/{case_id}/activity
GET /api/v1/cases/{case_id}/subscriptions
POST /api/v1/cases/{case_id}/subscriptions
DELETE /api/v1/cases/{case_id}/subscriptions/{id}

66. Suggested Database Tables

Additional tables introduced by v1.8:

search_documents
saved_searches

notifications
notification_preferences
notification_deliveries

calendar_events
calendar_connections
calendar_sync
calendar_attendees

subscriptions

comments
Reporting may initially use:

database_views
materialized_views
rather than introducing a separate reporting schema.

67. Search Index Data Model

A conceptual search document:

```
{
  "id": "uuid",
  "type": "document",
  "organization_id": "uuid",
  "case_id": "uuid",
  "title": "Contract",
  "content": "...",
  "metadata": {
    "document_type": "CONTRACT",
    "document_date": "2026-08-15"
  },
  "authorization": {
    "teams": ["team-id"]
  },
}
```

```
"updated_at": "2026-08-15T12:00:00Z"  
}
```

The actual index must not contain unnecessary sensitive information.

68. Audit Requirements

The following actions should be audited:

Search

For particularly sensitive environments, configurable search auditing may record:

```
user  
query type  
timestamp  
organization  
result count
```

Full search terms should only be stored where justified by organizational policy because search queries themselves may contain confidential information.

Notifications

Audit:

```
notification created  
notification delivered  
notification failed
```

Calendar

Audit:

```
event created  
event changed  
event deleted  
external synchronization
```

Reports

Audit sensitive report generation and exports.

Collaboration

Audit:

```
comments created  
comments changed  
subscriptions created
```

69. Data Retention

Search indexes, notifications and synchronization records are derived data and must follow defined retention policies.

Examples:

Old notifications:
 configurable cleanup

Search index:
 rebuildable from authoritative data

Calendar sync errors:
 retain operational history

Reports:
 generated exports follow document/export retention policy
Search data must never become the only surviving copy of a legal document or case record.

70. Security Requirements

All v1.8 services must enforce:

- authentication;
- authorization;
- organization isolation;
- object-level access control;
- TLS;
- secure service-to-service communication;
- audit logging;
- rate limiting;
- input validation;
- output encoding;
- secure external integration credentials.

External calendar OAuth credentials must be stored securely, preferably through the secret-management architecture defined in v1.1/v1.5.

71. Privacy Requirements

The system must minimize confidential information in:

- notification previews;
- email;
- push messages;
- logs;
- metrics;
- search indexes;
- external calendars;
- report filenames;
- URLs;
- browser telemetry.

The principle should be:

Store and transmit the minimum information required for the function.

72. Air-Gapped Deployment

The v1.8 architecture must remain compatible with fully local deployments.

Core functionality must not depend on:

Google

Microsoft

OpenSearch Cloud

external email services

external AI services

External integrations must be adapters.

An air-gapped deployment should be capable of operating with:

PostgreSQL

local search

local object storage

internal SMTP

internal identity provider

internal calendar infrastructure
internal message broker

73. Observability

Metrics should include:

Search

search_requests_total
search_latency
search_errors
index_lag
index_failures

Notifications

notifications_created
notifications_delivered
notification_failures
delivery_latency

Calendar

calendar_sync_total
calendar_sync_failures
calendar_sync_latency

Reports

report_requests
report_generation_time
report_failures
export_jobs

Sensitive search queries and case contents must not be placed in normal application logs.

74. Testing Requirements

Search tests

Test:

- Icelandic characters;
- case-insensitive matching;
- exact case number;

- full-text search;
- authorization filtering;
- confidential cases;
- deleted documents;
- stale indexes;
- reindexing.

Notification tests

Test:

- correct recipient;
- authorization;
- duplicate prevention;
- retries;
- delivery failure;
- preference handling;
- overdue notifications.

Calendar tests

Test:

- timezone conversion;
- overlapping events;
- event creation;
- modification;
- deletion;
- synchronization;
- synchronization failure;
- external credential expiry.

Reporting tests

Test:

- authorization;
- filters;
- aggregation;
- date ranges;
- exports;
- large datasets.

Collaboration tests

Test:

- comments;
- subscriptions;
- case activity;
- concurrent modifications;
- access revocation.

75. Performance Requirements

Initial targets:

Search

P95 < 500 ms
for ordinary searches under expected production load.

Notifications

Notification creation should normally occur within seconds of the triggering event.

Calendar

Calendar queries should support normal month/week views without excessive database queries.

Reports

Interactive reports should normally return within:

< 5 seconds

Large reports should become asynchronous jobs.

Performance targets must be validated through load testing rather than treated as guaranteed values.

76. Frontend Requirements

The frontend should provide:

Global search

A persistent search field supporting:

Cases

Documents

People

Tasks

Deadlines

Hearings

Search page

Provide:

search field

filters

result type

sorting

pagination

saved searches

Notification center

Provide:

unread count

notification list

read/unread

priority

navigation to source

Calendar

Provide:

day/week/month

filters

case

team

event type

Reporting

Provide:

dashboard
filters
charts/tables
export

77. Accessibility

All search, notification, calendar and reporting interfaces must meet the accessibility baseline established by the overall ILCMS frontend architecture.

Particular attention should be paid to:

- keyboard navigation;
- screen readers;
- calendar navigation;
- notification announcements;
- sufficient non-color indicators;
- accessible tables;
- accessible charts;
- Icelandic text rendering.

78. Localization

All user-facing terminology must be externalized.

Examples:

Search	=	Leit
Notification	=	Tilkynning
Calendar	=	Dagatal
Deadline	=	Frestur
Task	=	Verkefni
Report	=	Skýrsla
Case	=	Mál
Document	=	Skjal
Hearing	=	Þinghald / málflutningur

Legal terminology should be reviewed by Icelandic legal professionals before production.

Machine-readable enumeration values remain stable English identifiers.

79. Failure and Degradation Strategy

Individual subsystems must fail independently.

For example:

Search unavailable

↓

Case management continues

External calendar unavailable

↓

ILCMS calendar continues

Email unavailable

↓

In-app notifications continue

Reporting database unavailable

↓

Transactional case management continues

No derived subsystem should prevent core case operations unless the operation explicitly requires that subsystem.

80. Acceptance Criteria

v1.8 is considered implemented when:

Search

- Global search works.
- Case-number search works.
- Full-text document search works.
- Icelandic UTF-8 search works.
- Structured filtering works.
- Search is authorization-aware.
- Saved searches work.
- Search indexes can be rebuilt.
- Search failures do not prevent core case operations.

Notifications

- In-app notifications work.
- Deadline notifications work.
- Task notifications work.
- Hearing notifications work.
- Notification preferences work.
- Duplicate notifications are prevented.
- Failed delivery is retried.
- Notification access is authorization-aware.

Calendars

- Personal calendar works.
- Team calendar works.
- Case calendar works.
- Hearings appear correctly.
- Deadlines appear correctly.
- Timezones are handled correctly.
- Calendar conflicts are detected.
- External calendar architecture is implemented.
- Synchronization failures do not affect authoritative data.

Reporting

- Case reports work.
- Task reports work.
- Deadline reports work.
- Workload reports work.
- Proceeding reports work.
- Reports respect authorization.
- Exports work.
- Large reports use background jobs.

Collaboration

- Case activity feed works.
- Comments/notes work.
- Subscriptions work.
- Real-time updates are optional and non-authoritative.
- Concurrency conflicts are detected.
- Access revocation is respected.

Security

- All endpoints enforce v1.5 authorization.
- Tenant isolation is tested.
- Sensitive notification content is minimized.
- External calendar credentials are protected.
- Report exports are audited.
- Sensitive search activity is appropriately audited.

81. Implementation Phases

Phase 1 — Search MVP

Implement:

PostgreSQL full-text search
Case search
Party search
Document metadata search
Authorization filters
Global search API
Search UI

Phase 2 — Notifications

Implement:

Notification database
In-app notifications

Deadline reminders
Task reminders
Hearing reminders
Notification preferences
Background workers

Phase 3 — Calendar

Implement:

Personal calendar
Case calendar
Team calendar
Calendar event API
Timezone handling
Conflict detection

Phase 4 — Reporting

Implement:

Case reports
Task reports
Deadline reports
Workload reports
Basic dashboards
CSV/XLSX export

Phase 5 — Collaboration

Implement:

Case activity
Comments
Subscriptions
Real-time UI updates
Optimistic concurrency

Phase 6 — External Integrations

Implement optional adapters for:

Microsoft 365
Google Calendar
CalDAV
Enterprise messaging

82. Future Enhancements

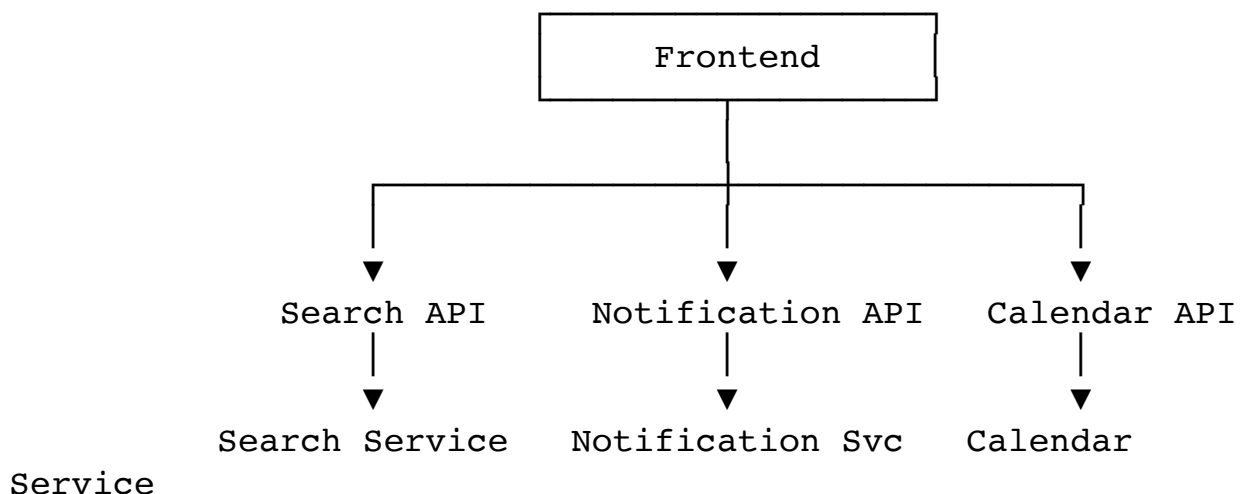
Potential future functionality:

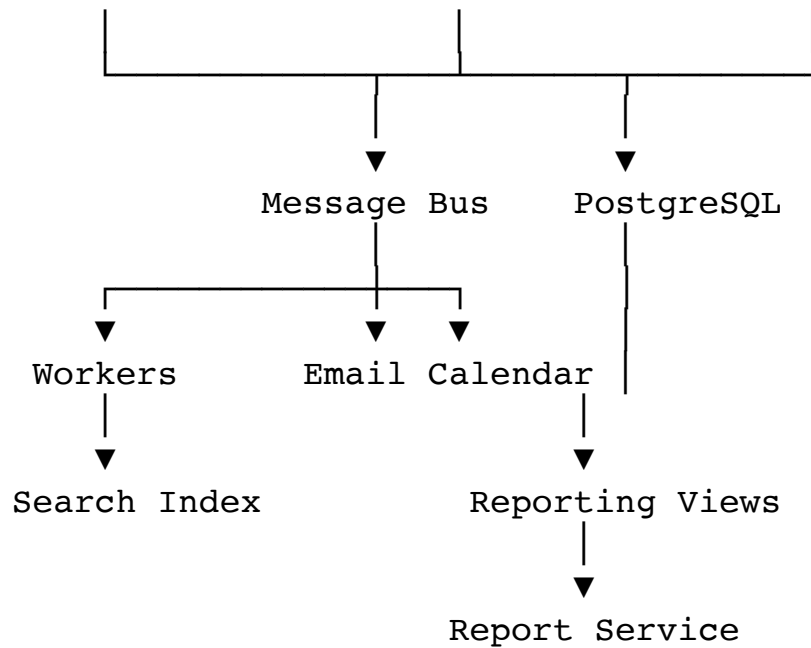
- semantic search;
- natural-language search;
- local/private AI-assisted search;
- advanced Icelandic legal-language search;
- predictive workload analysis;
- automated report generation;
- advanced calendar optimization;
- Microsoft Teams integration;
- collaborative document editing;
- real-time co-presence;
- dedicated analytics warehouse;
- business intelligence integration.

AI functionality must follow the AI governance principles established elsewhere in the ILCMS architecture and must not silently transmit confidential case information to external services.

83. Architectural Summary

The v1.8 architecture can be summarized as:





The architectural principle is:

PostgreSQL and the authoritative case/document records remain the source of truth. Search, notifications, calendars, reports and collaboration features are controlled, authorization-aware services built around that authoritative data.

84. Relationship to Previous Specifications

The major dependencies are:

```

v1.1  DevOps / Kubernetes
|
v1.3  Database
|
v1.4  REST API
|
v1.5  Authentication / Authorization
|
v1.6  Documents / Records
|
v1.7  Cases / Workflow / Tasks / Deadlines
|
▼
v1.8  Search / Notifications / Calendar /
      Reporting / Collaboration
  
```

v1.8 therefore completes much of the **operational user experience layer** around the core case-management model.

The next logical architectural areas are likely to be **v1.9 — Administration, Configuration, Master Data & System Management**, followed by a dedicated specification for **AI/LLM-assisted legal workflows and governance** if AI is to be a major component of ILCMS.

This gives us a coherent progression from the v1.7 case/workflow engine into the day-to-day user experience and operational management layer. The **v1.9 administration/configuration specification** would be a particularly useful next step because many of the things introduced in v1.7–v1.8—workflows, deadline rules, classifications, notification policies, calendars, permissions, templates, retention policies, etc.—need a proper administrative/configuration model.